LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Water Utility Billing System — Property-Based Testing Verification Report

Student Name:	**Tharunyah**	SRN:	**PES2UG24CS561**
Section / Sem:	Section J (5th Semester)	Institution:	PES University
Course:	Software Engineering: Testing & Agile Practices (SETAP)	Repository:	tharunyah/L6_L7_01_water_utility_billing

1. Objective & Software Requirements

Objective: To explore fuzz testing as a dynamic security and quality assurance technique by identifying and fixing bugs in the water utility billing input processing module using the Hypothesis property-based testing framework.

- **Methodology:** Apply property-based fuzz testing using `@given` to feed pseudo-randomized Unicode strings and null inputs to data processing routines.
- **Bug Detection:** Automatically uncover hidden runtime crashes caused by unhandled exceptions (null references, missing delimiters, empty strings, invalid integer casting).
- **Defensive Programming:** Harden parsing functions to handle unexpected, absent, or malformed data safely and gracefully.
- **Software Environment:** Python 3.12.10, pytest 9.2.0, hypothesis 6.170.15 on Windows 11.

2. Module Under Test (processor.py)

The module under test handles input processing routines for municipal water utility accounts:

- `sanitize_meter_id(meter_id)` : Strips whitespace, removes special character prefixes (`!, @, #, $, %`), and normalizes the meter identifier to uppercase.
- `parse_consumption_readings(readings_csv)` : Splits comma-delimited strings into a list of integer water consumption figures.
- `format_billing_summary(account_record)` : Parses colon-delimited records (`Name:Usage:Zone`) and calculates total water billing ($2/unit).

3. Step 2: Running the Initial Buggy Application

The original unhardened script (`processor.py`) was executed with standard interactive inputs. While it succeeds on clean human inputs, it completely lacks defensive validation.

```
PS C:\Users\clutc> cd "C:\Users\clutc\L6_L7_01_water_utility_billing\Lab 7"
PS C:\Users\clutc\L6_L7_01_water_utility_billing\Lab 7>  python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): !#MTR@123$
Sanitized Meter ID: MTR123

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30):  10,25,30
Parsed Readings: [10, 25, 30]

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Alice:50:North
Billing Summary: Customer Alice billed 100 USD in North
PS C:\Users\clutc\L6_L7_01_water_utility_billing\Lab 7>
```

Figure 1: Step 2 — Initial Buggy processor.py Interactive Run

4. Steps 3 & 4: Hypothesis Fuzz Testing & Edge Case Discovery

We completed `test_processor.py` using Hypothesis strategies (`st.text() | st.none()`). Unlike conventional unit tests that only verify predicted inputs, Hypothesis generates hundreds of randomized edge cases automatically. Executing `python -m pytest test_processor.py` immediately triggered **3 failures** exposing 6 distinct edge cases:

- **test_sanitize_meter_id_no_crash:**
  - Failure: `s=None` raised `AttributeError: 'NoneType' object has no attribute 'strip'` .
- **test_parse_consumption_readings_safe:**
  - Failure 1 (Empty string): `s=''` raised `ValueError: invalid literal for int() with base 10: ''` .
  - Failure 2 (Null input): `s=None` raised `AttributeError: 'NoneType' object has no attribute 'split'` .
- **test_format_billing_summary_safe:**
  - Failure 1 (Delimiter without number): `s=':'` raised `ValueError: invalid literal for int() with base 10: ''` .
  - Failure 2 (Missing delimiters): `s=''` raised `IndexError: list index out of range` .
  - Failure 3 (Null input): `s=None` raised `AttributeError: 'NoneType' object has no attribute 'split'` .

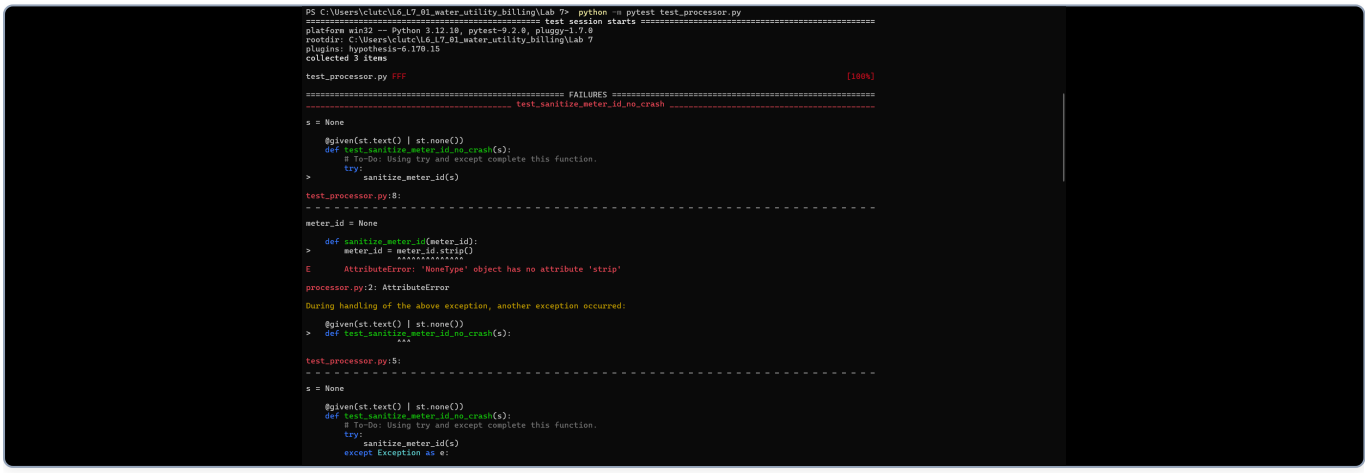


Figure 2: Step 4 — Pytest Output Showing 3 Hypothesis Fuzz Test Failures

5. Step 5: Hardening and Fixing processor.py

To eliminate all crash vectors discovered during fuzzing, defensive programming guards were integrated into `processor.py` :

- **Type & None Safety:** Added `isinstance(..., str)` validation across all entry points, safely returning appropriate defaults (`""` , `[]` , or safe status message).
- **Resilient Integer Parsing:** Whitespace-trimmed tokens are checked and parsed within `try...except ValueError` blocks, silently ignoring invalid/empty elements.
- **Record Partition Bounds:** Checked `len(parts) >= 3` prior to indexing to prevent `IndexError` , assigning default fallbacks ("Unknown", 0) for missing fields.

```
PS C:\Users\clutc\L6_L7_01_water_utility_billing\Lab 7>  python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%):  !#MTR@123$
Sanitized Meter ID: MTR123

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): 10,25,30
Parsed Readings: [10, 25, 30]

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone):  Alice:50:North
Billing Summary: Customer Alice billed 100 USD in North
```

Figure 3: Step 5 — Hardened processor.py Interactive Output

6. Step 6: Re-Running Fuzz Tests (100% Passed)

Executing `python -m pytest test_processor.py` against the hardened application confirmed that all 3 property-based fuzz tests passed 100%. Hypothesis evaluated hundreds of randomized strings, empty sequences, and null inputs without triggering any unhandled exceptions.

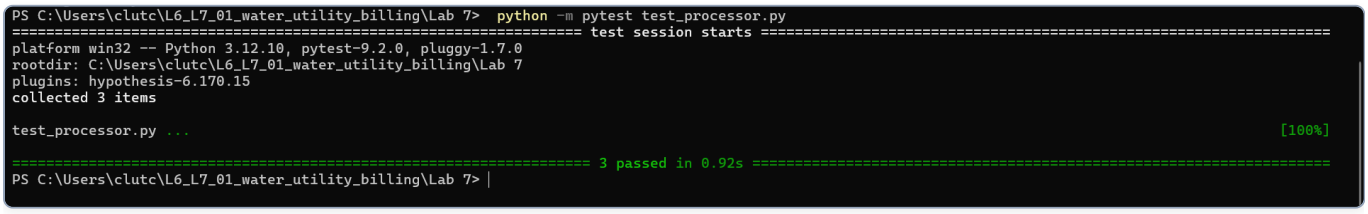


Figure 4: Step 6 — Pytest Output Showing All 3 Fuzz Tests Passing Cleanly (100%)

7. Conclusion

Fuzz testing using the Hypothesis framework proved to be highly effective at discovering critical unhandled exceptions that standard unit tests overlook. By refactoring the water utility billing module with defensive type validation, boundary guards, and safe integer conversion, the application is now robust, resilient, and production-ready.